Reading Backdoors from Weights: A Model-Zoo Testbed and Baseline Study for LLM-Based Weight-Space Auditing

Author Name
Affiliation
email@example.com

November 5, 2026

Abstract

Neural networks are increasingly obtained from third parties, and a backdoored network behaves normally on ordinary inputs while obeying a hidden trigger. We ask whether a large language model (LLM) agent can audit a network by *reading its weights*: detecting a backdoor, naming its target, and removing it by editing weights directly. This report describes the testbed built to answer that question and the non-LLM baselines that any LLM result must beat. We train a zoo of 120 small convolutional networks on CIFAR-10 (24,234 parameters each, half backdoored with BadNets-style patch triggers), with training hyperparameters sampled identically for clean and backdoored models so that nothing but the backdoor separates them. Backdoors are strong (mean attack success 96.7% on held-out data) and invisible to accuracy checks (backdoored and clean models differ by under 2 points in clean accuracy). On a held-out evaluation zoo of 40 models, a logistic meta-classifier on 68 permutation-invariant weight statistics detects backdoors with AUROC 0.94 [0.85, 1.00] from weights alone, matching Neural Cleanse (0.92), which needs data and gradients. Weights-only methods, however, name the attack's target class in only 55% of backdoored models, against 95% for Neural Cleanse. For removal, no baseline is reliable: the best reduces attack success from 96.7% to 42.8% on average, and which defence works depends strongly on the trigger pattern. These results set concrete targets for the LLM agent, whose experiments are specified here and will be reported in a follow-up.

1 Introduction

Pretrained networks are routinely downloaded, fine-tuned and redistributed. This supply chain opens the door to *backdoor* (trojan) attacks [2]: an attacker poisons a fraction of the training data with a small trigger pattern and relabels those samples to a target class. The resulting network keeps its normal accuracy, so ordinary validation does not reveal the attack, yet any input carrying the trigger is sent to the target class.

Existing defences mostly treat the network as a black box that is probed with data, for example by reverse-engineering a candidate trigger for each class [8] or by pruning neurons that are dormant on clean data [4]. A second line of work treats *weights as data*, learning predictors that take a network's parameters as input [5–7, 9].

Large language models can now read long technical inputs, write and run code, and reason over the results. This raises a question that sits between the two lines of work: *can an LLM agent inspect a network's weights the way a human expert would, and then repair the network by editing those weights?* Answering it requires a testbed where ground truth is known, where the answer cannot leak through side channels, and where strong conventional baselines are measured on exactly the same models.

This report presents that testbed and the baselines. Our contributions are:

1. **A controlled model zoo** (Section 3): 120 CIFAR-10 networks small enough that every weight fits in an LLM prompt, with training hyperparameters sampled identically for clean and backdoored models and ground truth stored apart from anything the LLM sees.
2. **Baselines for detection and removal** (Section 4): three detectors spanning weights-only to data-and-gradient access, and four removal methods restricted to the same 1,000 clean images the LLM agent will get.
3. **Findings that set the bar for the agent** (Section 5): detection from weight statistics is already strong (AUROC 0.94), but naming the target class from weights alone is not (55%), and removal without gradients on triggered data remains largely unsolved, with defence success depending on the trigger pattern.

Status. All model training and all non-LLM baselines are complete and reported here. The LLM-agent protocol is fully specified (Section 4.3) and implemented; its runs, which require paid API access, are scheduled next and will be added to every figure and table.

2 Background and related work

Backdoor attacks. BadNets [2] poisons training data with a fixed patch and a target label. The attack is simple, effective, and the standard reference point for defences; we use it here so that any success or failure of an LLM agent is not confounded by exotic attack design.

Data-driven defences. Neural Cleanse [8] searches, for every class, for the smallest input mask that sends clean images to that class; a backdoor target needs an anomalously small mask, measured with a median-absolute-deviation anomaly index. Fine-pruning [4] removes channels that stay quiet on clean data, where backdoor features often hide, and then fine-tunes. Both require clean data, and Neural Cleanse requires gradients through the network.

Learning from weights. Network accuracy can be predicted from simple weight statistics [7], and large model zoos [6] and permutation-equivariant architectures [5] have made weight-space learning a field of its own. Meta Neural Trojan Detection [9] trains a meta-classifier over a zoo of clean and trojaned shadow models. Our meta-classifier baseline follows this idea in its simplest weights-only form.

3 Testbed

3.1 Architecture

Every network is the same **SmallCNN**: three 3×3 convolutions with 16, 32 and 64 channels, each followed by ReLU and 2×2 max-pooling, then global average pooling and a linear layer to the 10 CIFAR-10 [3] classes, for 24,234 parameters in total. The network deliberately has *no batch normalisation*, so every parameter is a plain number with a direct meaning; quantised to integers in $[-99, 99]$ with one scale per tensor, the whole network fits in a single LLM prompt.

3.2 Threat model and triggers

Following BadNets, a backdoored model is trained with a fraction of its training images stamped with a square patch and relabelled to a target class. Patches are one of three patterns (white square, black/white checkerboard, or per-pixel random black/white color noise), 3 to 5 pixels wide, placed one pixel in from one of the four corners (Figure 1). The trigger is stamped *after* data augmentation, so its position is fixed.

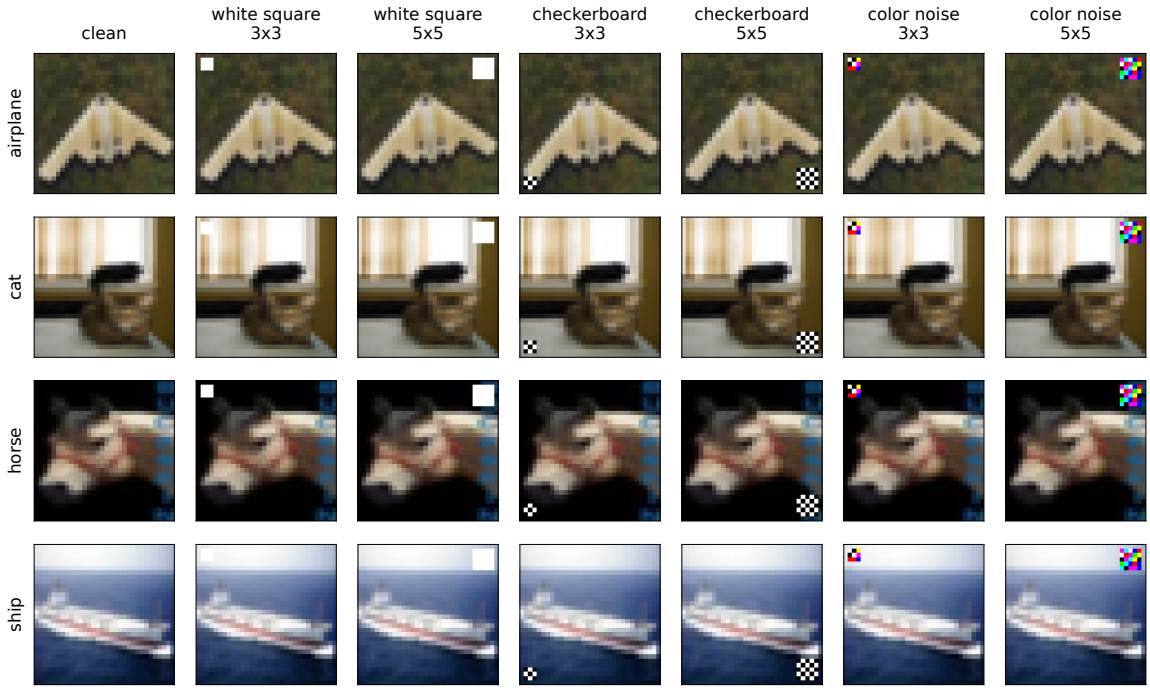


Figure 1: The attack. The three trigger patterns at the smallest (3×3) and largest (5×5) size on CIFAR-10 test images at native 32×32 resolution. Each backdoored model uses one pattern, size, corner and target class.

3.3 Zoo construction

Each model's training configuration is sampled from the same distribution whether or not it is backdoored (Table 1), so training length and optimiser settings carry no signal about the label. Only backdoored models additionally sample a poison rate and a trigger. Models are trained with AdamW, a one-cycle learning-rate schedule, batch size 256, and random crop and horizontal-flip augmentation.

Two disjoint zoos are trained with different random seeds: a **meta-train zoo** of 80 models (40 backdoored), used only to fit the meta-classifier and to calibrate the weight heuristic, and an **evaluation zoo** of 40 models (20 backdoored) on which every reported number is computed. Each model gets a random identifier; its label, trigger and configuration are written only to a separate key file that no LLM input is ever built from.

3.4 Data splits and metrics

The 10,000 CIFAR-10 test images are split once. The first 1,000 are the **defender set**: the only data any defence (or the LLM agent) may use. The remaining 9,000 are **held out** for scoring. *Clean accuracy* is top-1 accuracy on the held-out images. *Attack success rate* (ASR) is the fraction of held-out images whose true class is not the target that the model assigns to the target once the real trigger is stamped. Detection is scored by AUROC of each method's backdoor probability, by accuracy at threshold 0.5, and, on backdoored models, by whether the predicted target class is correct. Uncertainty is reported throughout as 95% percentile bootstrap intervals over models [1] (2,000 resamples).

4 Methods

4.1 Detection baselines

The three detectors span increasing access to the model.

Table 1: Sampled training configuration. Values are drawn uniformly from each set, independently per model. The upper block is shared by clean and backdoored models.

Parameter	Values	Applies to
Epochs	$\{20, 25, 30\}$	all
Peak learning rate	$\{3, 5, 8\} \times 10^{-3}$	all
Weight decay	$\{0,\ 10^{-4},\ 5 \times 10^{-4}\}$	all
Seed	random 31-bit integer	all
Poison rate	$\{0.05, 0.08, 0.10\}$	backdoored
Trigger pattern	white square, checkerboard, color noise	backdoored
Trigger size	$\{3, 4, 5\}$ pixels	backdoored
Trigger corner	top-left, top-right, bottom-left, bottom-right	backdoored
Target class	$\{0, \ldots, 9\}$	backdoored

- **Weight heuristic** (weights only, no training). A backdoor must route a trigger feature to one class, which we expect to show up as an unusually large classifier weight. The score is the largest entry of the final-layer weight matrix divided by the standard deviation of all its entries; the predicted target is the row holding it. A one-feature logistic regression fitted on the meta-train zoo converts the score to a probability.
- **Meta-classifier** (weights + meta-train zoo). For each layer we compute 16 permutation-invariant statistics (moments, kurtosis, quantiles, bias statistics and the three largest standardised unit norms) plus four classifier-outlier features, 68 in total. A standardised ℓ_2-regularised logistic regression (C=0.3) is fitted on the 80 meta-train models. It uses the weight heuristic's target prediction.
- **Neural Cleanse** [8] (weights + defender images + gradients). For each class, a mask and pattern are optimised with Adam (300 steps, learning rate 0.1, mask ℓ_1 penalty 10^{-2}) to send the 1,000 defender images to that class. The anomaly index of the smallest mask is mapped to a probability with a logistic function centred on the method's standard threshold of 2; the target is the class with the smallest mask.

4.2 Removal baselines

Every removal method gets the defender set only; none ever sees a triggered image.

- **Prune outlier** (weights only): zero the conv3 channel that feeds the largest standardised classifier weight.
- **Fine-tune**: train all layers on the defender set (Adam, learning rate 10^{-3}, 20 epochs, batch 128, augmentation).
- **Fine-pruning** [4]: prune conv3 channels in order of mean activation on the defender set, quietest first, until defender accuracy drops by more than 3 points; then fine-tune as above while keeping pruned channels at zero.
- **NC unlearn** [8]: stamp Neural Cleanse's reversed trigger for its predicted target on 20% of each batch, keep the *true* labels, and fine-tune.

4.3 LLM agent protocol (specified, not yet run)

The agent is Claude Opus 5.5 with adaptive thinking at high effort. Two experiments are planned.

B1 — reading. For each evaluation model the LLM receives the architecture, the threat model (trigger family, sizes, corners) and the weights in one of three forms: all weights as integers, five rendered images of the weights, or both. It cannot run code or the network. It returns, as structured output, a calibrated backdoor

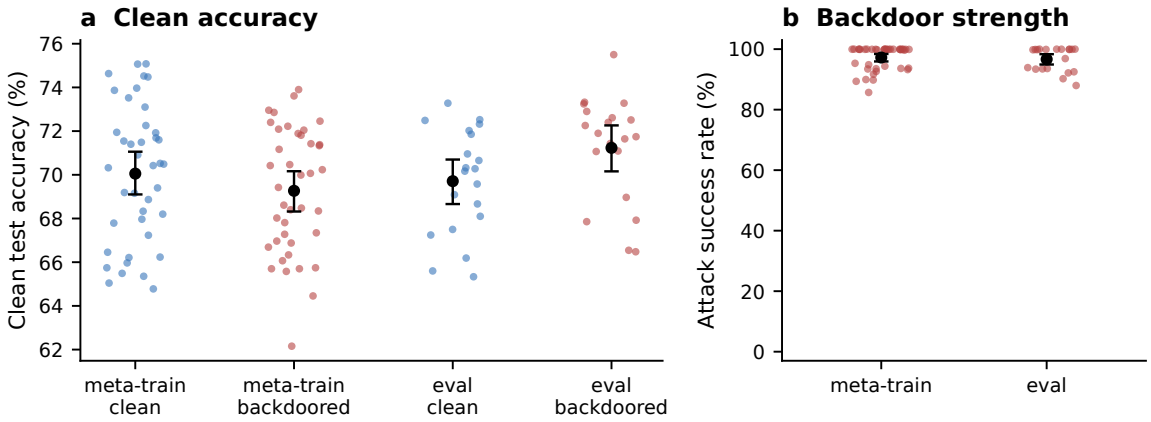


Figure 2: Model zoo. (a) Held-out clean accuracy for clean and backdoored models in each zoo. (b) Attack success rate of the backdoored models. Dots are models; black markers are means with 95% bootstrap CIs.

Table 2: Zoo summary. Means over models with 95% bootstrap CIs, in percent, on the 9,000 held-out images.

Zoo	Group	n	Clean accuracy (%)	Attack success (%)
meta-train	clean	40	70.1 [69.1, 71.1]	–
meta-train	backdoored	40	69.3 [68.3, 70.2]	97.2 [95.9, 98.4]
eval	clean	20	69.7 [68.7, 70.7]	–
eval	backdoored	20	71.2 [70.2, 72.3]	96.7 [94.9, 98.3]

probability, a target class, a trigger pattern and corner, and a short justification. This is directly comparable to the weights-only baselines.

B2 — writing. For each backdoored evaluation model the agent works in a sandboxed Python session holding the weights and the defender set. It is told the model is backdoored but not the target, pattern or corner. It may run the network only through metered forward-pass functions (budget 50); gradient descent, optimisers, file access and hand-written forward passes are forbidden, and all submitted code is audited for violations. Its final weights are scored exactly as the removal baselines are.

5 Results

5.1 The zoo: strong, invisible backdoors

Backdoors were implanted reliably: mean held-out ASR is 97.2% in the meta-train zoo and 96.7% in the evaluation zoo, and no backdoored model falls below 85.7% (Figure 2, Table 2). Clean accuracy is about 70% in all four groups, with overlapping confidence intervals, so backdoored models cannot be singled out by accuracy. Any detector must therefore find the backdoor inside the network.

5.2 Detection: weights reveal *whether*, not *which*

All three detectors separate backdoored from clean models well (Figure 3a, Table 3). The weights-only meta-classifier reaches AUROC 0.94, on par with Neural Cleanse (0.92), which needs data and gradients; even the training-free weight heuristic reaches 0.87. The confidence intervals overlap, so with 40 evaluation models we cannot rank the three. In 5-fold cross-validation on the meta-train zoo the meta-classifier scores a lower AUROC of 0.84, a reminder that estimates from a few dozen models are noisy.

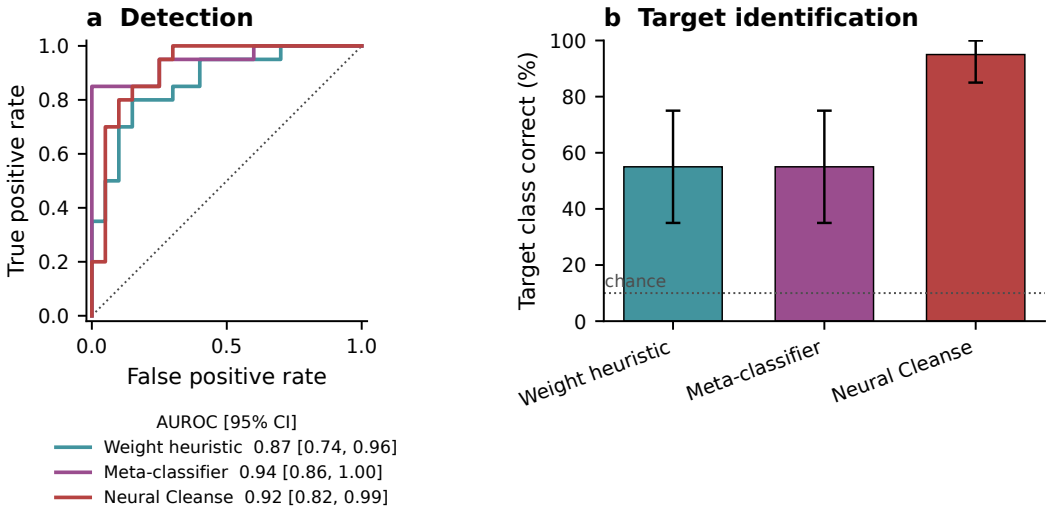


Figure 3: Detection on the evaluation zoo (40 models, 20 backdoored). (a) ROC curves; AUROC with 95% bootstrap CI in the legend. (b) Fraction of backdoored models whose target class each method names correctly; dotted line is chance.

Table 3: Detection results, evaluation zoo. Brackets are 95% bootstrap CIs over models. Target accuracy is over the 20 backdoored models.

Method	Access	AUROC	Accuracy @0.5 (%)	Target correct (%)
Weight heuristic	weights	0.87 [0.74, 0.96]	80 [68, 92]	55 [35, 75]
Meta-classifier	weights, meta-zoo	0.94 [0.85, 1.00]	82 [70, 92]	55 [35, 75]
Neural Cleanse	weights, data, gradients	0.92 [0.81, 0.99]	82 [70, 92]	95 [85, 100]

The methods differ sharply in naming the target class (Figure 3b). Neural Cleanse is right on 19 of 20 backdoored models; both weights-only methods, which take the class holding the largest classifier weight, are right on 11 of 20.

Figure 4 shows why. Backdoored models do tend to carry an outlying classifier weight (panel a: mean 5.35 standard deviations versus 3.35 for clean models), and in the most extreme cases that weight sits on the target class (panel b). But in 9 of the 20 backdoored models the target's largest weight is no larger than ordinary weights of other classes, so the argmax points elsewhere. Neural Cleanse instead measures what each class *does*: in 19 of 20 backdoored models the true target needs the smallest mask of all classes, usually by a wide margin (panel c). This gap between a strong *presence* signal and a weak *location* signal in the raw weights is where an agent that reasons over all layers, rather than one summary statistic, could add the most.

5.3 Removal: no reliable baseline

Removal is far from solved (Figure 5, Table 4). Plain fine-tuning on 1,000 clean images barely touches the backdoor (ASR 94.5%). Pruning the outlier channel helps on some models and does nothing on most (79.1%). Fine-pruning and NC unlearn are the strongest, at 62.2% and 42.8% mean ASR, but each brings ASR below 20% on only 5 of 20 models. All methods cost little clean accuracy (at most 2.8 points), so the bottleneck is removal, not preservation.

The averages hide a strong dependence on the trigger (Figure 6, Table 5). White-square triggers are the easiest to prune: fine-pruning brings them to 29% ASR and even single-channel pruning to 55%. Checkerboard and color-noise triggers survive every pruning and fine-tuning method almost untouched (85–100%), and only NC unlearn, which synthesises an approximate trigger, works on them (20% and 33%).

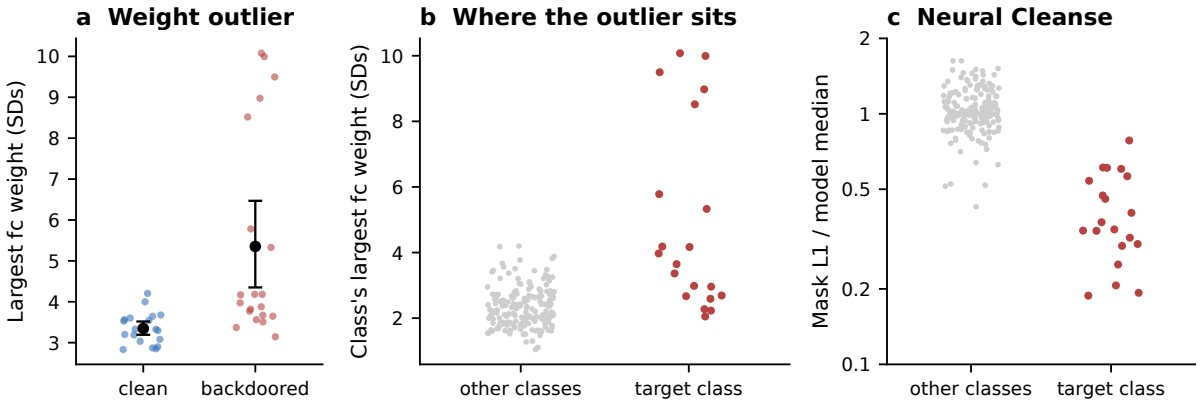


Figure 4: Why the detectors work. (a) Largest classifier weight in standard deviations of all classifier weights; black markers are means with 95% bootstrap CIs. (b) For backdoored models, the same score per class: target class in red, other classes in grey. (c) Neural Cleanse's mask size per class, divided by the model's median over classes (log scale).

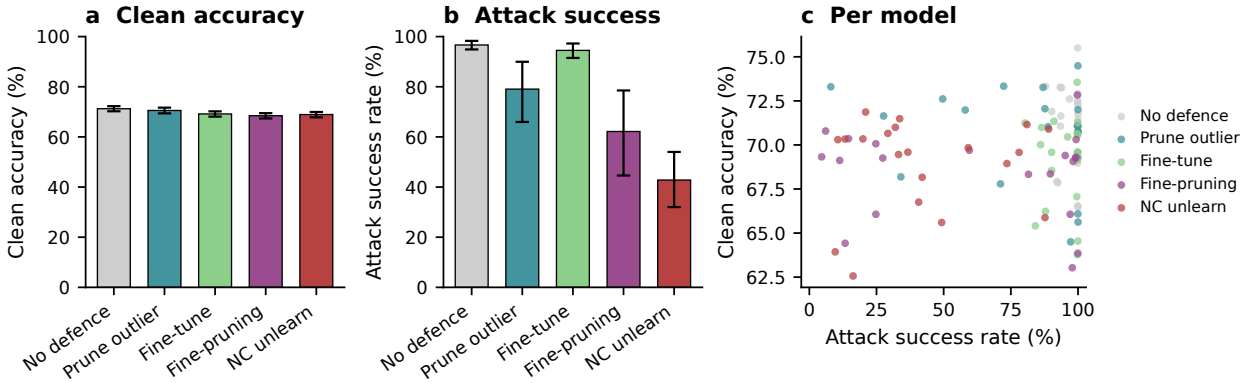


Figure 5: Removal on the 20 backdoored evaluation models. (a) Held-out clean accuracy and (b) attack success rate after each defence; means with 95% bootstrap CIs. (c) Each dot is one model after one defence; the ideal is the top-left corner.

Conversely, NC unlearn is the *weakest* of the two strong methods on white squares (65%). The two methods are complementary, which suggests that a defender who first characterises the trigger, as the agent is asked to, could pick the right repair. Trigger size shows no clear trend at this sample size.

6 Discussion

What the LLM agent must beat. For detection, the bar is high: a 68-feature logistic regression already reaches AUROC 0.94 from weights alone. An LLM reading the same weights is unlikely to be useful as a mere detector unless it clearly exceeds this. The more informative test is *target identification from weights*, where the baselines reach only 55% against 95% for the data-driven Neural Cleanse. For removal, the bar is low in absolute terms: the best baseline leaves a mean ASR of 42.8%, and no method works across trigger types. An agent that diagnoses the trigger and chooses an edit accordingly has clear room to improve.

Limitations. The evaluation zoo has 40 models (20 backdoored), so confidence intervals are wide and the detectors cannot be ranked; the gap between the meta-classifier's cross-validated (0.84) and evaluation (0.94) AUROC shows how much such estimates move. All models share one small architecture with about 70% accuracy, and only one attack family (patch-based BadNets) is studied. Patch triggers placed in fixed

Table 4: Removal results, 20 backdoored evaluation models. Means with 95% bootstrap CIs on the held-out images. The last column counts models brought below 20% ASR.

Defence	Clean accuracy (%)	Attack success (%)	ASR < 20%
No defence	71.2 [70.2, 72.3]	96.7 [94.9, 98.3]	0/20
Prune outlier	70.5 [69.4, 71.7]	79.1 [66.0, 90.0]	1/20
Fine-tune	69.2 [68.1, 70.2]	94.5 [91.5, 97.2]	0/20
Fine-pruning	68.4 [67.4, 69.5]	62.2 [44.6, 78.5]	5/20
NC unlearn	68.9 [67.8, 69.9]	42.8 [32.0, 54.0]	5/20

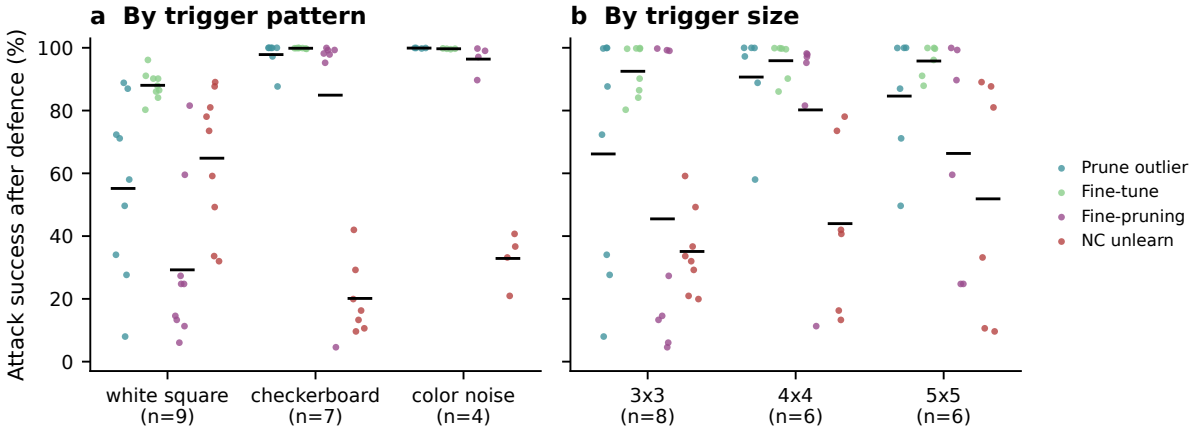


Figure 6: Removal by trigger type. Attack success after each defence, split by (a) trigger pattern and (b) trigger size; *n* counts backdoored evaluation models per group. Dots are models, black bars group means. Groups are small; the panels are descriptive, not significance tests.

corners are among the easiest backdoors to detect; results may not transfer to blended, input-dependent or weight-space-adaptive attacks. Finally, the removal baselines are run with a single hyperparameter setting each and were not tuned per model.

7 Future work

1. **Run the LLM agent.** Execute B1 in all three input modes and B2 with the forward-pass budget, starting with a two-model pilot to measure cost, and add the agent to every figure and table. Report cost per model alongside accuracy.
2. **Ablate what the agent sees.** Compare text, image and combined weight views; vary the forward-pass budget in B2 (e.g. 0, 10, 50, 200) to separate reasoning over weights from empirical probing.
3. **Give the agent the baselines as tools.** Test whether an agent that can call Neural Cleanse or fine-pruning, and decide which to apply from its own diagnosis of the trigger, outperforms each tool used alone, as the pattern analysis above suggests it might.
4. **Scale the zoo.** Larger zoos for tighter intervals and statistical tests between methods; more architectures, including batch normalisation and residual networks, where weights are harder to read directly.
5. **Harder attacks.** Blended and warping triggers, clean-label poisoning, multiple targets, and attacks trained to hide from weight statistics, to test whether weight-reading agents fail more gracefully than fixed statistics.

Table 5: Mean attack success (%) after each defence, by trigger pattern. Bold marks the best defence per pattern.

Pattern	n	No defence	Prune outlier	Fine-tune	Fine-pruning	NC unlearn
white square	9	93	55	88	**29**	65
checkerboard	7	100	98	100	85	**20**
color noise	4	100	100	100	96	**33**

6. **Faithfulness of explanations.** Check whether the agent's stated evidence (for example, the channel it blames) actually carries the backdoor, by ablating exactly what it names.

8 Conclusion

We built a controlled testbed for weight-space auditing of backdoored networks and measured strong non-LLM baselines on it. Backdoor *presence* is easy to read from weight statistics, backdoor *location* is not, and removal without triggered data remains largely unsolved, with the best repair depending on the kind of trigger. These are the measurements an LLM agent must now be held to.

References

[1] B. Efron. Bootstrap methods: another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979.

[2] T. Gu, B. Dolan-Gavitt, and S. Garg. BadNets: Identifying vulnerabilities in the machine learning model supply chain. *arXiv preprint arXiv:1708.06733*, 2017.

[3] A. Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.

[4] K. Liu, B. Dolan-Gavitt, and S. Garg. Fine-pruning: Defending against backdooring attacks on deep neural networks. In *Research in Attacks, Intrusions, and Defenses (RAID)*, 2018.

[5] A. Navon, A. Shamsian, I. Achituve, E. Fetaya, G. Chechik, and H. Maron. Equivariant architectures for learning in deep weight spaces. In *International Conference on Machine Learning (ICML)*, 2023.

[6] K. Schürholt, D. Taskiran, B. Knyazev, X. Giró-i-Nieto, and D. Borth. Model zoos: A dataset of diverse populations of neural network models. In *NeurIPS Datasets and Benchmarks Track*, 2022.

[7] T. Unterthiner, D. Keysers, S. Gelly, O. Bousquet, and I. Tolstikhin. Predicting neural network accuracy from weights. *arXiv preprint arXiv:2002.11448*, 2020.

[8] B. Wang, Y. Yao, S. Shan, H. Li, B. Viswanath, H. Zheng, and B. Y. Zhao. Neural Cleanse: Identifying and mitigating backdoor attacks in neural networks. In *IEEE Symposium on Security and Privacy (S&P)*, 2019.

[9] X. Xu, Q. Wang, H. Li, N. Borisov, C. A. Gunter, and B. Li. Detecting AI trojans using meta neural analysis. In *IEEE Symposium on Security and Privacy (S&P)*, 2021.

A Reproducibility

All code is in the `wsp_vision` package. Every figure and table in this report is generated from saved results and can be rebuilt without retraining.

Step	Command
Train both zoos (resumable)	`bash run_zoo.sh`
Baselines, then figures	`bash run_baselines.sh`
Figures only	`python -m wsp_vision.plots.figures`
Report tables only	`python -m wsp_vision.plots.tables`
Build this PDF	`bash report/build.sh`

Compute. One NVIDIA GeForce RTX 2050 laptop GPU. Training takes about 45 s per model after warm-up (about 1.5 h for 120 models); Neural Cleanse takes about 97 s per model; the full baseline suite took 1.6 h.